

CHAPTER 6

Arithmetic and Logic Operations

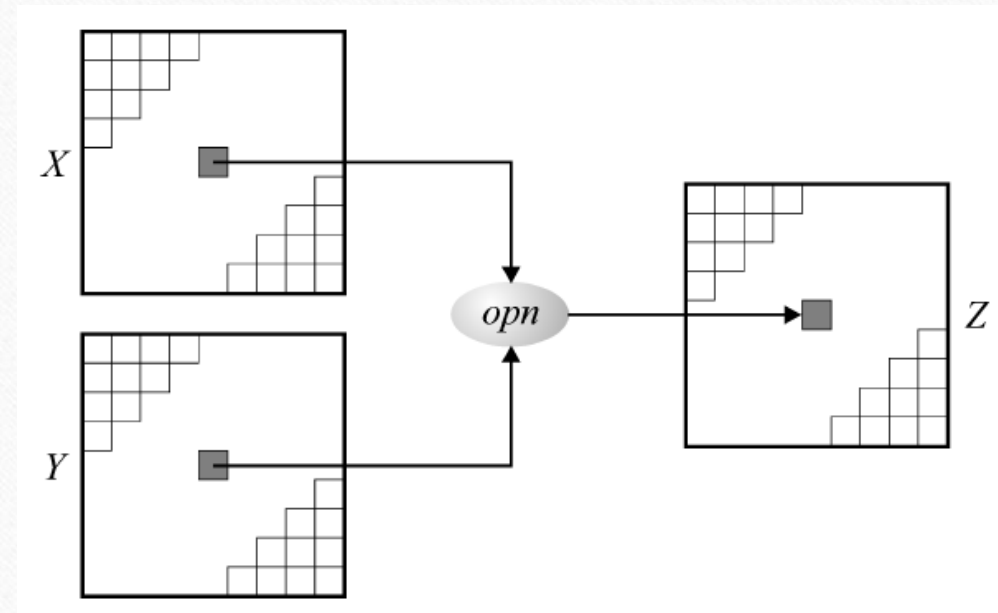
WHAT WILL WE LEARN?

- Which arithmetic and logic operations can be applied to digital images?
- How are they performed in MATLAB?
- What are they used for?

Arithmetic and logic operations

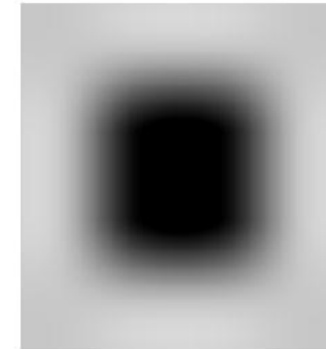
- Combine two images, pixel-by-pixel, using an arithmetic or logical operator, resulting in a third image, Z:

$$X \text{ } opn \text{ } Y = Z$$



Addition

- Used to blend the pixel contents from two images or to add a constant value to an image's pixel values.
- Example 1 (blending two images): $Z = X + Y$



Addition

- Example 3 (additive noise):

$$Z = X + N$$



(a)



(b)



(c)

Addition

- Example 2 (additive image offset):

$$Z = X + 75$$



Addition in MATLAB

- `imadd` function (see Tutorial 6.1)
- Handling overflow:
 - Truncation
 - Normalization

$$g = \frac{L_{max}}{f_{max} - f_{min}}(f - f_{min})$$

Addition in MATLAB

- Handling overflow (Example 6.1):

```
X = uint8([200 100 100; 0 10 50; 50 250 120])
```

```
Y = uint8([100 220 230; 45 95 120; 205 100 0])
```

```
W = uint16(X) + uint16(Y)
```

```
fmax = max(W(:))
```

```
fmin = min(W(:))
```

```
Za = uint8(255.0*double((W-fmin))/double((fmax-fmin)))
```

```
Zb = imadd(X,Y)
```


- `im = imread('pout.tif');`
- `imshow(im)`
- `im = double(im);`

- `im = im/max(im(:))*255;`
- `sigma=10;`
- `n = randn(size(im)) * sigma;`
- `imnoise = im + n;`
- `figure; imshow(imnoise/255)`

Subtraction

- Used to detect differences between two images, decrease its overall brightness, or obtain its negative.
- Example 1 (subtractive image offset): $Z = X - 75$



Subtraction

- Example 2 : negative of an image



Subtraction in MATLAB

- Three functions:

- `imsubtract`
- `imabsdiff`
- `imcomplement`

- Example 6.2:

```
X = uint8([200 100 100; 0 10 50; 50 250 120])
```

```
Y = uint8([100 220 230; 45 95 120; 205 100 0])
```

```
Za = imsubtract(X,Y)
```

```
Zb = imsubtract(Y,X)
```

```
Zc = imabsdiff(Y,X)
```

Multiplication and division in MATLAB

- Functions:
 - `immultiply`
 - `imdivide`
- Example (Fig. 6.6):



Logic operations

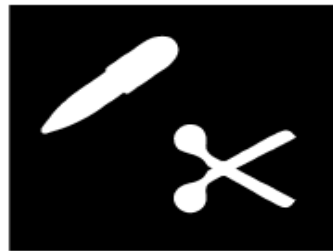
- Performed in a bit-wise fashion on the binary contents of each pixel value.
 - AND, XOR and OR require two or more arguments.
 - NOT operator only requires only one argument.
- Book examples use the convention: 1 (true) for white pixels; 0 (false) for black pixels.

Logic operations

- Examples using binary test images



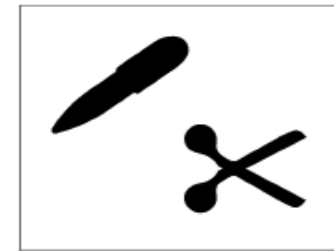
X



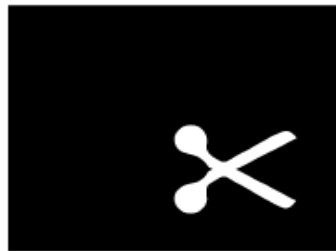
Y



$\text{NOT } X$



$\text{NOT } Y$



$X \text{ AND } Y$



$X \text{ OR } Y$



$X \text{ XOR } Y$



$(\text{NOT } X) \text{ AND } Y$

Logic operations on monochrome images

- Example 1 (AND)



Logic operations on monochrome images

- Example 2 (OR)



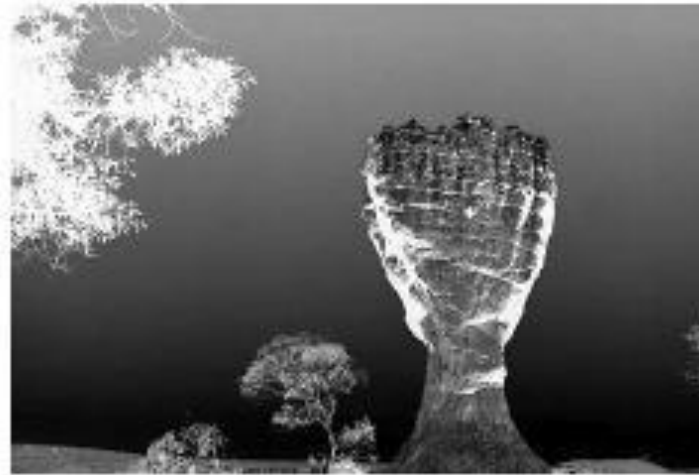
Logic operations on monochrome images

- Example 3 (XOR)



Logic operations on monochrome images

- Example 4 (NOT)



Logic operations in MATLAB

- Functions (see Tutorial 6.2):
 - **bitand**
 - **bitor**
 - **bitxor**
 - **bitcmp**

Hands-on



- Tutorial 6.1: Arithmetic operations (page 113)
- Tutorial 6.2: Logic operations and region of interest (ROI) processing (page 118)